Rapport d'optimisation — TP R3.14

Étudiant : Soane ILOAI — BUT MMI Béziers
Site déployé : https://soane986.github.io/R314-optimisation-site/
Fork (historique des commits) : https://github.com/Soane986/R314-optimisation-site
Site de référence (avant) : https://nico3807.github.io/R314-optimisation-site/

> Les champs [à compléter] sont à remplir avec mes propres relevés, dont les captures se trouvent dans le dossier `audit/`.

---

1. Conditions de mesure

Paramètre	Valeur
Navigateur	Chrome [version à compléter], fenêtre de navigation privée (pas d'extensions)
Onglet Réseau	« Désactiver le cache » coché, rechargement Ctrl+Maj+R
Lighthouse (DevTools)	Catégorie Performance, mode Navigation, profils Mobile **et** Desktop
Panneau Performances	CPU ×4, enregistrement d'au moins 20 s avec défilement, fenêtre de plus de 1024 px puis de moins de 1024 px
PageSpeed Insights	Site **déployé** (jamais localhost) : 3 analyses par profil, valeur médiane retenue
Avant	Site de référence (non modifié)
Après	Mon fork publié sur GitHub Pages

Limite méthodologique : PageSpeed Insights n'affiche aucune donnée de terrain (rapport CrUX), car le site n'a pas assez de trafic réel. L'audit repose donc entièrement sur des données de laboratoire, c'est-à-dire des conditions simulées.

Poids en local et en ligne : les fichiers texte (HTML, CSS, JS) sont servis compressés en gzip par GitHub Pages, mais pas par un serveur local. Les poids ci-dessous sont les tailles des fichiers sur le disque. Les poids transférés relevés dans l'onglet Réseau du site en ligne sont donc plus faibles pour le texte, et identiques pour les images.

---

2. Tableau comparatif des six indicateurs

Indicateur	Avant — Mobile	Après — Mobile	Avant — Desktop	Après — Desktop
Poids total transféré	16,8 Mo	[à compléter]	16,8 Mo	[à compléter]
Requêtes	18	[à compléter]	18	[à compléter]
FCP	[à compléter]	[à compléter]	[à compléter]	[à compléter]
LCP	[à compléter]	[à compléter]	[à compléter]	[à compléter]
CLS	[à compléter]	[à compléter]	[à compléter]	[à compléter]
TBT	[à compléter]	[à compléter]	[à compléter]	[à compléter]
Score PSI (médiane de 3)	[à compléter]	[à compléter]	[à compléter]	[à compléter]

Poids des fichiers propres au site (taille sur disque, hors polices Google)

Ressource	Avant	Après	Gain
6 photos de la galerie	9,92 Mo (+ img6 téléchargée 2 fois : 3,39 Mo)	139 Ko (en 400 px) à 498 Ko (en 800 px)	−96 à −99 %
Texture du bandeau (`hero-bg`)	3,39 Mo	37 Ko	−98,9 %
Favicon	787 Ko	2 Ko	−99,7 %
CSS (`styles` + `extra` + `theme-legacy`)	122 Ko, 3 fichiers en chaîne	5,2 Ko, 1 fichier	−95,7 %
JS bloquant dans le `<head>`	3 scripts (dont un 404)	0 (`app.js` et `analytics.js` en `defer`)	—
Total de la page d'accueil	**≈ 17,6 Mo (16,8 Mio)**	**≈ 0,21 Mo (écran 1x) à 0,57 Mo (écran 2x)**	**−97 à −99 %**
Requêtes vers le site (hors Google)	17, dont un 404 et un doublon	12, dont 3 images différées (`lazy`)	−5

---

3. Problèmes identifiés et corrections

Les numéros de ligne renvoient au code **d'origine**. Chaque correction correspond à un commit du fork.

Niveau 1 — visibles dans l'onglet Réseau et Lighthouse

#	Fichier et ligne	Nature	Preuve	Coût	Correction (commit)
1	`index.html:75-90` , `img1-6.html`	Photos en 2040 à 3968 px affichées dans des cartes d'environ 388 px ; aucun `width` / `height` , aucun `lazy`	Réseau trié par taille ; infobulle « dimensions intrinsèques / taille rendue » ; `audit/avant-reseau.png`	9,9 Mo d'images	Variantes de 400, 800 et 1200 px (JPEG q78), `srcset` / `sizes` calculés à partir des media queries, `width` / `height` (CLS), `loading="lazy"` sur la 2e rangée, `fetchpriority="high"` sur la 1re vignette — *Redimensionne et compresse les photos*
2	`img1-6.html`	Image des pages de détail en taille native, qui fait défiler la page horizontalement	Mode appareil mobile (Ctrl+Maj+M)	Lisibilité, et jusqu'à 3,4 Mo	Règle `.image-wrapper img{max-width:100%;height:auto}` — même commit
3	`index.html:12`	`preload` de `img6.jpg?v=2` : URL différente de `img6.jpg` , donc **téléchargée deux fois** ; de plus, c'est une image hors de l'écran	Deux lignes `img6.jpg` dans l'onglet Réseau	3,39 Mo, et de la bande passante prise aux ressources critiques	Préchargement supprimé — même commit
4	`css/theme-legacy.css:11`	Texture de 3968 px (3,4 Mo, même fichier que img6) affichée à **4 % d'opacité**	Image présente dans l'onglet Réseau mais introuvable à l'écran	3,39 Mo	Version 640 px de 37 Ko, rendu identique à cette opacité (effet conservé) — *Allège favicon et texture hero*
5	`index.html:8`	Favicon PNG de 512 px (787 Ko)	Onglet Réseau	787 Ko	PNG de 32 px (2 Ko) — même commit
6	`index.html:16`	`js/legacy/polyfills.js` en **404**, script **bloquant** dans le `<head>`	Filtre « statut ≠ 200 », console	Un aller-retour bloquant pour rien	Référence supprimée — *Supprime préchargements et requêtes inutiles*

#	Fichier et ligne	Nature	Preuve	Coût	Correction (commit)
7	`index.html:9`	`dns-prefetch` vers `cdn.jsdelivr.net`, jamais contacté	Ce domaine n'apparaît pas dans l'onglet Réseau	Une résolution DNS inutile	Supprimé — même commit
8	`index.html:10-11`	`preload` de police **sans** `crossorigin`, qui pointe sur la graisse 300 inutilisée	Avertissement de la console (« credentials mode does not match »)	Police téléchargée deux fois	Supprimé — même commit
9	`css/styles.css:1-2`, `extra.css:1`, `theme-legacy.css:3`	Chaîne de 4 `@import` en série (dont Google Fonts deux fois)	Cascade « en escalier » dans l'onglet Réseau	4 allers-retours avant le premier rendu	Un seul fichier CSS ; Poppins chargée par `<link>` avec `preconnect` — *Fusionne CSS, polices non bloquantes*
10	`css/styles.css:2`	Police en `display=block` (texte invisible au chargement) et graisse 300 inutilisée	Lighthouse : « affichage des polices »	FCP retardé	`display=swap`, graisses 400 et 600 seulement (le gras 700 reste rendu en 600, comme avant) — même commit
11	`css/styles.css` (1199 règles), `extra.css` (800 règles), `theme-legacy.css`	Règles `.redundant-*`, `.extra-*` (`filter`), `.tile`, `.legacy-*` jamais utilisées	Onglet Coverage ; aucune occurrence dans les fichiers HTML et JS	117 Ko de CSS bloquant	Supprimées : 122 Ko → 5 Ko — *Supprime règles CSS jamais utilisées*

Niveau 2 — visibles seulement dans un profil d'exécution

#	Fichier et ligne	Nature	Preuve	Coût	Correction (commit)
12	`js/vendor/ui-kit.min.js` (code déplié)	Au chargement : boucle `while` de **620 ms** sur `performance.now()` et création de **240 000 objets** jamais utilisés	Panneau Performances : bloc rouge juste après le téléchargement du script ; Bottom-Up → `c` dans `ui-kit.min.js`	≈ 620 ms de TBT (bien plus avec CPU ×4)	Librairie supprimée : `app.js` n'utilisait que `qs`, `qsa`, `cls` et `ready`, remplacés par l'API native — *Remplace UIKit par JavaScript natif*
13	`js/vendor/ui-kit.min.js`	`MutationObserver` sur tout le document (`subtree`, `attributes`) qui appelle `getComputedStyle()` sur **tous** les éléments à chaque mutation	Bottom-Up : `_sync` / `getComputedStyle` pendant le défilement	Multiplié par le n°14 : chaque style écrit au scroll relance l'observateur	Supprimé avec la librairie — même commit
14	`js/app.js:157-188` (`revealCards`)	Appelée à **chaque** événement scroll et resize : `querySelectorAll` et lecture de `offsetHeight` de toutes les cartes **à l'intérieur** de la boucle sur les cartes ($O(n^2)$), lectures et écritures de style alternées (**reflow forcé**), puis `getBoundingClientRect` sur les **371** cases du calendrier	Avertissement « Forced reflow » sur les blocs violets du panneau Performances	Des dizaines de reflows par événement scroll	`IntersectionObserver` pour les cartes ; hauteurs égalisées en lisant tout puis en écrivant tout, au chargement et au resize seulement ; calendrier : 1 lecture et 7 variables CSS (une par ligne), écouteur `passive` limité par `requestAnimationFrame`. Opacités vérifiées identiques (écart max $4 \cdot 10^{-7}$) — *Supprime reflows forcés au scroll*

#	Fichier et ligne	Nature	Preuve	Coût	Correction (commit)
15	`js/app.js:81` (`buildKeywordIndex`)	`innerHTML +=` dans une boucle de 320 mots-clés : toute la liste est re-analysée à chaque tour (coût quadratique)	Bottom-Up : « Parse HTML » sous `buildKeywordIndex`	51 360 options créées au lieu de 320	`DocumentFragment` et une seule insertion — *Construit suggestions avec fragment unique*
16	`index.html:17-18`, `img1-6.html`	`ui-kit.min.js` et `app.js` chargés sans `defer` dans le `<head>` ; sur les pages de détail, ils ne servent à rien	Lighthouse : « ressources qui bloquent le rendu »	Analyse du HTML bloquée	`app.js` en `defer` ; scripts retirés des pages de détail — *Remplace UIKit par JavaScript natif*

Niveau 3 — conditionnel

#	Fichier et ligne	Nature	Preuve	Coût	Correction (commit)
17	`js/analytics.js:19-29` (code obscurci)	Les chaînes base64 décodées donnent `innerWidth`, `performance`, `now` et `hidden`. **6,5 s après le chargement**, toutes les **2,6 s**, si l'onglet est visible et la fenêtre **> 1024 px** : boucle `while` d'attente active de 46 ms, **allongée de 28 % à chaque passage** jusqu'à **430 ms**	Enregistrement Performance de 20 s en fenêtre large : blocs rouges réguliers et de plus en plus longs. Invisibles en profil mobile ou sur un enregistrement court, d'où l'écart entre les mesures	Long tasks croissantes : la page « rame après quelques secondes », exactement la plainte du client	Script **conservé** (même déclenchement, mêmes conditions, même objet `window.__au`), mais empreinte du DOM calculée une seule fois et plus de boucle d'attente — *Supprime boucle bloquante analytics desktop*

---

4. Optimisations majeures : problème, correction, gain mesuré

Optimisation	Indicateur visé	Avant	Après	Gain mesuré
Images (n°1, 3, 4, 5)	Poids, LCP	16,8 Mo	[à compléter]	[à compléter]
Chaîne critique CSS et polices (n°6 à 11)	FCP, requêtes	[à compléter]	[à compléter]	[à compléter]
Suppression de UIKit (n°12, 13, 16)	TBT	[à compléter]	[à compléter]	[à compléter]
Reflows au scroll (n°14)	Fluidité du scroll (durée des tâches dans le panneau Performances)	[à compléter]	[à compléter]	[à compléter]
`analytics.js` (n°17)	Long tasks après 6,5 s en desktop	46 → 430 ms toutes les 2,6 s	[à compléter]	[à compléter]

Remarque : si une optimisation n'a rien changé de mesurable, je l'indique ici. C'est aussi un résultat.

---

5. Contrôle de l'iso-rendu

Captures avant/après : `audit/iso-avant-desktop.png`, `audit/iso-apres-desktop.png` (et la même chose en mobile).
Points vérifiés :

- Même mise en page, mêmes couleurs, mêmes ombres. La texture du bandeau, l'animation d'arrivée des cartes et le fondu du calendrier sont conservés.
- Filtre par mot-clé, liste de 320 suggestions, calendrier de 371 jours et fenêtre des consignes fonctionnent tous.

- Seule différence volontaire : l'image des pages de détail ne déborde plus de l'écran (c'était un défaut, cf. guide A1).
- Le flou `backdrop-filter` de la barre de navigation est coûteux à afficher, mais il fait partie du rendu : il est conservé.

---

6. Optimisations impossibles sur GitHub Pages

Ces optimisations relèvent du serveur, que je ne contrôle pas sur un hébergement statique :

- **Cache-Control** : `max-age=31536000, immutable` sur les images, le CSS et le JS, et une revalidation courte sur le HTML. GitHub Pages impose `max-age=600`.
- **Empreinte dans les noms de fichiers** (`styles.a3f9c1.css`) pour pouvoir mettre un cache long sans risquer de servir une version périmée. Cela demande un outil de build, interdit ici.
- **Brotli** au lieu de gzip pour les fichiers texte (environ 15 à 20 % de mieux).
- **Polices hébergées sur le site** : cela supprimerait deux domaines tiers (googleapis et gstatic) et les connexions associées. Je ne l'ai pas fait ici, faute de pouvoir télécharger les fichiers `.woff2` sans outil.
- **WebP / AVIF** avec `<picture>` : environ −30 % (WebP) à −50 % (AVIF) supplémentaires sur les photos. C'est faisable hors ligne avec Squoosh, mais pas réalisé faute d'encodeur sur ma machine.
- **CDN** pour rapprocher les fichiers des visiteurs, et **HTTP/2 ou HTTP/3** (déjà fournis par GitHub Pages).

---

7. Conclusion

[à compléter : synthèse des gains mesurés, ce qui a le plus payé (les images pour le poids, UIKit et analytics pour le TBT), et ce que j'ai appris sur la différence entre audit court et audit long.]